
DEVELOPMENT DOCUMENTATION

3. Process Walkthrough

The pipeline, step by step, documents in to posts out

Dynamic Auctioneers Marketing Platform

Built by Keegan Haumann, Cognexa

Compiled 20 August 2026 · code state **9c9e0df** on main

The full journey of one property, from documents arriving to adverts live and changes handled. This is the document to read to understand what the system actually does.

At every step: **what the human does**, **what the system does**, and **what has to be true to move on**.

Step 0 — Before the platform

The documents the team already produces arrive in the ordinary way. Nothing about their habits had to change, which was a deliberate design constraint.

DOCUMENT	WHO PRODUCES IT	WHAT THE SYSTEM TAKES FROM IT
Lightstone EVM	Bought in	Deeds data, identity, market valuation, comparables
Property Report	Gerrie, by inspection	Physical reality, features, terms of sale, photographs
Valuer's report	Sometimes present	Professional market and forced-sale values. Optional
OTP / Conditions of Sale	The instruction	The sale terms the information pack prints
Levy statement	Managing agent	The monthly levy figure

Only the **EVM and the Property Report** gate completeness. The other three are optional and never hold a job up.

Step 1 — Intake (M1)

Human: opens the platform, goes to Intake, and drags the documents onto the drop zone. Up to three PDFs in the main slots, plus photographs.

System:

1. Reads the text of each PDF with PyMuPDF and scores it against five weighted marker sets — `lightstone_evm`, `property_report`, `valuation_report`, `otp`, `levy_statement`. **Content decides the classification; the filename is only a tiebreaker**, so a badly named file still lands correctly.
2. Resolves the DP number from the filename or folder (`3060 - ...`, `3035.1 - ...`). A sub-property resolves to its parent plus a lot number. If nothing carries a DP number, the screen **asks for it** rather than guessing (D50).
3. Merges multiple files or portions into **one combined record** (D50) — a multi-portion property is not several records.
4. Builds an `IntakeJob` and checks completeness.

To move on: both the EVM and the Property Report must be present. A lone document **parks visibly** — the screen says "property report missing" — rather than proceeding on half the facts.

Then: an `extract` job is queued and the record enters state `intake`.

Step 2 — Extraction (M2)

Runs on the background worker. The team watches progress on the intake screen; nothing blocks the browser.

System:

1. **Sectioned extraction** (D21). Rather than one enormous call, the model is asked for one section at a time, each returning validated structured output. The same six-tool list `record_<section>` is sent on every call so the prompt prefix is identical and **caching actually hits** — a per-call difference had been silently invalidating it, and fixing it made extraction about three times cheaper (D32).
2. **Non-strict tool use** (D23), because a strict grammar rejected even a flat section as too complex.
3. PDFs go in as **native document blocks**. A text-input mode with call pacing exists as an opt-in fallback from when the rate limit was 10,000 input tokens per minute; the tier is 500,000 now, so neither is needed (D22, D31).
4. **Photographs** are pulled from the Property Report at source quality with PyMuPDF. These are often low-resolution thumbnails, and the uploader warns about it without blocking (D25).
5. **The OTP is read** for its clauses and **the levy statement** for the monthly figure (D68, D73, D75), both by rebuilding table rows from word positions so the reader behaves identically on the server (D77).
6. **Source precedence and conflicts**. Physical facts resolve valuation > property report > lightstone in code. Every disagreement is stored as a structured conflict with each source's value.
7. **The merge rule**. Lightstone wins deeds and market data; the inspection wins physical reality; a conflict becomes a flag, never a silent pick.
8. **Normalisation** (D30) canonicalises dates, title type and zoning afterwards.

Guarantees: the schema forbids fields it does not know about, so an invented field is rejected outright. A fact the documents do not contain is `null`, never guessed. Owner name and ID land in the internal layer only.

Output: a validated record in state `extracted`, written to SQLite and to `DP<dp>/record.json`, with photographs in `DP<dp>/photos/`.

Cost: about R5, and free on a repeat because the call is content-addressed cached (D59).

Step 3 — Verification and Gate 1 (M3)

What the system checks

Deterministic cross-checks, in code, no model, no key: extent, title deed, municipal valuation, GPS, and bedroom, bathroom and garage counts across the sources.

Live web research, through the model's server-side search tool: comparable listings in the suburb, a recent-sales sanity check, and whether the address exists. On the golden property this ran 11 searches and found an active listing at the same address at R995,000 — inside the EVM range — plus the useful detail that the portals call the suburb "Pelham", not "Pelham North" (D31).

The memo

`verification-memo.md`: corroborated facts, numbered flags each with evidence and an action, market context, and a POPIA checklist. Flags are severity-typed:

- `block` — stops publishing until resolved or overridden with a written reason
- `note` — internal awareness only

The memo carries findings, not the model's workings (D31).

Gate 1 — the human

The approver opens the verification screen and sees the flags. For each physical conflict there is a **source picker**, defaulting to the precedence winner, which can be overridden to take another source's value (D35). Blocking flags must be resolved or explicitly overridden with a reason. Then they sign off.

Nothing reaches `drafted` without sign-off. Sign-off is the only path into `verified`, and it is state-guarded in code.

This step earns its keep. It caught the DP3060 garages unprompted — Lightstone said three, the inspection found none — and the flatlet that existed only in the inspection. On the second property it caught a two-versus-four bedroom and a 106-versus-310 m² disagreement that were previously being resolved silently.

Step 4 — Photographs

An explicit step between gate 1 and the advert draft (D47, D52), because rendering an advert without photographs wastes the render.

Human: uploads photographs, sets the lead photograph, replaces or removes any of them, and checks them in a lightbox (D76).

System: every drop **adds** to what is already chosen — a second folder appends rather than replacing (D70). The cap is **40 per property** (raised from 8, D70). Photographs extracted from the source PDF are offered as a fallback. Low-resolution files are badged with their pixel dimensions as a warning, not a block. The canonical `marketing.hero_photo` and `marketing.gallery` are written on the record, and the serving route is authentication-gated.

Because humans manage photographs here, the OneDrive/Graph watcher stopped being a dependency and became an optional convenience (D20).

Step 5 — The advert and Gate 2 (M5)

The heart of the daily work.

The draft

The system generates the advert from the record: copy by the model, layout by the brand templates. Copy is channel-aware, South African English, framed by `sale_process.method` — "offers invited" versus auction — with no em dashes and no AI tells. Every claim traces to a record field.

What the marketer can change on gate 2

CONTROL	BEHAVIOUR
Design	Four designs: <code>hero_overlay</code> (default), <code>stats_first</code> , <code>feature_list</code> , <code>collage</code> . Between them they cover every layout in the team's own 55 adverts
Headline	Typed, or an Auto-generate button (D38)
Copy and fields	The small-edits editor. Edits are stored on the record as <code>human_overrides</code> and applied last in <code>public_view()</code> , so they reach every artifact and survive re-renders (D17)
Photographs	Upload, reorder by promoting a lead, replace in place, remove, lightbox (D76)
QR code	Supplied by the team and uploaded; the platform prompts for it (D69)
Auction fields	Editable auction detail. The board's weekday is <i>derived from the auction date</i> , never typed (D71)
Template set	Where more than one named Canva design set is configured and the renderer routes through Canva, a set can be picked per property (D33)

Edits batch. Nothing re-renders on every keystroke or save: changes are held and a single explicit **Regenerate** does the render (D72). A pending-render marker tracks it, and that marker is excluded from anything the client downloads (D77).

The advert previews in a same-origin iframe, and exports as an emailable PNG at 2160×2700 (D39, D48).

Gate 2 — approval

The artifact set is emailed to `admin@dynamicauktioneers.co.za` carrying **signed, single-use, expiring links**. An approver can approve or request changes **from the email alone, without logging in**.

A change request regenerates the artifacts and returns to gate 2 in one screen flow — it does not fall back to the start.

Step 6 — Gate 3, client approval

System: drafts the client email.

Human: sends it. This stays manual **by the team's choice**, not by limitation. When the client replies happy, the reply is logged in the platform with a date and a user.

After this point, change requests are internal only. The client is not re-consulted (D13). This mirrors what the team already did.

Step 7 — The artifact set is built

Only after the client says yes does the full set get produced — nine artifacts from the one record:

ARTIFACT	FORMAT	NOTES
<code>demo_ad</code>	HTML, PNG, PDF	1080×1350, rasterised 2×
<code>info_pack</code>	PDF	Paginated A4 landscape , rebuilt to the team's own packs. Terms read from the OTP's clauses, not hardcoded — every value that had been hardcoded was wrong (D68). Every page is filled: what it cannot fill with content it fills with photographs (D60)
<code>auction_board</code>	PDF	Rebuilt to the team's own board (D74). Boards and billboards are two separate artefacts, confirmed by the owner
<code>portal_listing</code>	Markdown	Property24-ready
<code>facebook_post</code>	Markdown	Post plus boost notes
<code>email_blast</code>	Markdown	Subject A/B plus body
<code>alert_mailer</code>	HTML	Plus the audience
<code>saia_banner</code>	HTML	SAIA alert banner
<code>webapp_icon</code>	SVG	Website tile

Everything is written to `DP<dp>/artifacts/` with a `manifest.json` logging the pass. The artifacts screen shows each one as **a preview of itself**, not a grey icon (D56), and offers download-all. That screen doubles as the Proof of Marketing view.

Step 8 — Distribution (M6)

System: computes the channel matrix from the record's value and type (section 2.6), then:

- **Facebook, Instagram, LinkedIn** post through the GoHighLevel Social Planner. Photographs and the advert PNG are uploaded to the GHL Media Library and attached as CDN URLs (D30).
- **Everything else** gets a ready-to-post pack with a checklist.
- Posted or not-posted is tracked **per channel per version**.

Human: chooses save-as-draft, schedule, or post now (D28) — subject to the guard rail.

The guard rail: `GHL_POST_STATUS=draft` in the server environment **overrides any choice made in the UI**. A misconfigured box cannot publish to a live page. Posting to a real page requires deliberately changing it on the server.

Static images only; video is out of scope (D24).

Step 9 — Changes after going live

A price change reaches all nine artifacts in one pass (D64) — a fault found by driving the real application rather than by a unit test, which is worth remembering about how this system gets verified.

The flow:

1. The record is edited. Its state moves to `updated`.
2. Every artifact regenerates.
3. Fast-path re-approval — internal only, the client is not re-consulted.
4. API channels re-post automatically; manual channels get a fresh pack.
5. **A price drop additionally queues a "REDUCED" re-engagement burst** — a marketing event, not silent maintenance.

Price changes post a visible "REDUCED" update rather than editing a post silently, because a silent edit wastes the news.

Step 10 — Buyer CRM seed (M7)

Every enquiry — a "reply 3060" message, an email link carrying `?dp=3060`, a Facebook lead — creates or updates a contact tagged with the DP number and a derived category (residential or industrial, area, budget band).

A new verified listing queries the matched buyers and produces a targeted broadcast: "new industrial property in Jet Park, 214 matched buyers".

Deliberately minimal. It seeds the full Buyer CRM phase of the wider engagement rather than trying to be it.

What stays manual, and why

Four things, each on purpose:

1. **The approval clicks at the three gates.** By design. The system drafts, humans approve.
2. **Sending and receiving the client email.** The team's choice.
3. **Printing and erecting the auction boards.** Physical.

4. **Deleting already-live Instagram posts.** No delete API exists. GHL's own delete only tidies its planner. A possible future mitigation is publishing on a 15–30 minute delay so a recall button can work before a post goes live.

Everything else automates.

The as-is workflow, and what the system now covers

The team's workflow before the platform, and the verdict on each step:

#	STEP AS IT WAS	NOW
1	Documents arrive	Automated — intake and extraction
2	Advert built by hand from Canva templates	Automated — brand-token templates recreating the designs
3	Advert emailed to <code>admin@</code> for internal approval	Routing automated , the click stays human — approve-by-email links
4	Approved advert emailed to the client	Manual by choice — the system drafts it
5	Client replies happy	Semi — a human logs the reply
6a	Website icon or tile	Automated
6b	SAIA banner plus alert mailer	Mostly — banner, HTML and audience generated; scheduling depends on the GHL API
6c	Information pack	Automated . The paid Meta campaign stays a human boost
6d	Auction boards	Design automated , printing and erecting are physical
6e	Post to all platforms	Automated through GHL
7	Change request loop	Automated — the <code>updated</code> fast path
8	Delete and repost	Partial — regenerate and repost are automated; deleting live Instagram posts is manual